

MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 2 2025/2026

WEEK 9: REMOTE MONITORING SYSTEM AND CONTROL VIA BLUETOOTH

SECTION 1

GROUP 9

INSTRUCTOR: DR WAHJU SEDIONO & ASSOC. PROF. EUR. ING. IR. TS. GS. INV.

DR ZULKIFLI BIN ZAINAL ABIDIN

NO	NAME	MATRIC NO
1	NOR EFFENDI BIN ANUAR @ ANWAR	2310519
2	NUR AINA SAFFIYA BINTI MOHD TARMIZI	2411644
3	NUR ATIKAH BINTI KAMARUL BAHARIN	2319846

DATE OF SUBMISSION: 13 MAY 2026

TABLE OF CONTENTS

1.0 INTRODUCTION.....	3
2.0 MATERIALS AND EQUIPMENT.....	3
3.0 EXPERIMENTAL SETUP.....	4
4.0 METHODOLOGY.....	5
4.1 Circuit Setup.....	5
4.2 Program Development and Uploading.....	5
4.3 Program Operation.....	5
4.4 Coding.....	6
5.0 DATA COLLECTION.....	8
6.0 DATA ANALYSIS.....	8
7.0 RESULTS.....	9
8.0 DISCUSSION.....	9
9.0 CONCLUSION.....	10
10.0 RECOMMENDATIONS.....	10
11.0 REFERENCES.....	11
ACKNOWLEDGEMENT.....	12
STUDENTS' DECLARATION.....	12

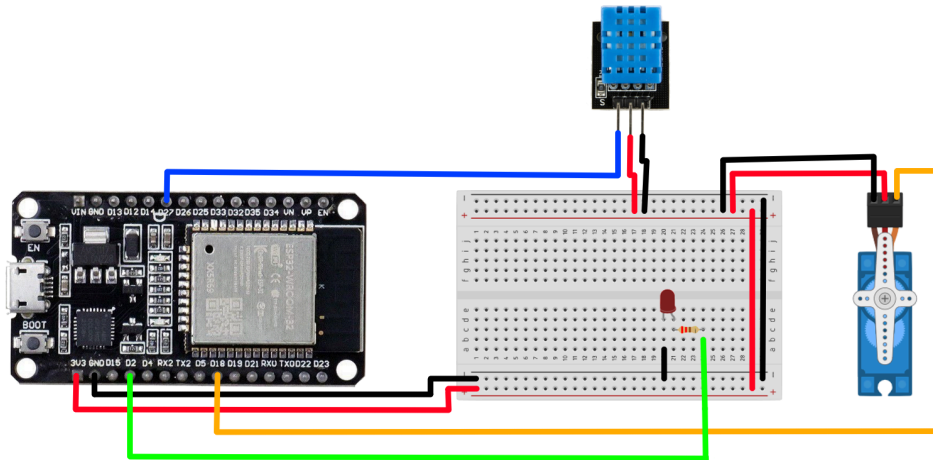
1.0 INTRODUCTION

The integration of Bluetooth technology into Mechatronics and IoT systems allows for seamless, wireless data exchange and remote control between hardware and smart devices. By utilizing the ESP32's built-in 2.4 GHz radio frequency and Frequency-Hopping Spread Spectrum (FHSS), reliable communication is maintained even in environments with potential interference. This project demonstrates a remote monitoring and control system where an ESP32 acts as a wireless bridge, transmitting real-time sensor data to a smartphone while simultaneously receiving commands to actuate physical components, such as servos.

2.0 MATERIALS AND EQUIPMENT

- ESP32 development board
- Temperature sensor (DHT11)
- Servo Motor
- Smartphone (Android or IOS)
- Power supply for the ESP32/Arduino
- Breadboard
- Jumper wires

3.0 EXPERIMENTAL SETUP



1. Connection between the ESP32 and Breadboard:
 - a. Connect the 3V3 ESP32 pin to the positive terminal of the breadboard.
 - b. Connect the GND ESP32 pin to the negative terminal of the breadboard.
2. Connection between the ESP32, Breadboard and DHT11:
 - a. Connect the positive pin of DHT11 to the positive terminal of the breadboard.
 - b. Connect the negative pin of DHT11 to the negative terminal of the breadboard.
 - c. Connect the signal pin of DHT11 to the GPIO27 pin on the ESP32.
3. Connection between the ESP32, Breadboard and Servo Motor:
 - a. Connect the positive pin of the servo motor to the positive terminal of the breadboard.
 - b. Connect the negative pin of the servo motor to the negative terminal of the breadboard.
 - c. Connect the signal pin of the servo motor to the GPIO18 pin on the ESP32.
4. Connection between the ESP32, Breadboard and LED:
 - a. Connect the positive leg of the LED to the leg of 220 Ω resistor.
 - b. Connect the other leg of the 220 Ω resistor to the positive terminal of the breadboard.
 - c. Connect the negative leg of the LED to the negative terminal of the breadboard.

4.0 METHODOLOGY

4.1 Circuit Setup

- Connect the data pin of the sensor (DHT22) to a designated GPIO pin on the ESP32. Ensure the sensor is powered via the 3.3V or 5V pin and grounded to the ESP32.
- Connect the servo motor's signal wire to a PWM-capable GPIO pin. Provide external power if the servo requires more current than the ESP32 can supply, ensuring a common ground between the power source and the ESP32.
- Use a breadboard and jumper wires to organize the connections as per the original circuit logic, replacing the standalone potentiometer with the new sensor-servo configuration.

4.2 Program Development and Uploading

- Include the `BluetoothSerial.h` library for communication and the `ESP32Servo.h` (or standard `Servo`) library to handle motor control.
- Initialize the Bluetooth serial interface using `SerialBT.begin("Device_Name")`.
- Define the logic to read the sensor value using `analogRead()` or the specific sensor library.
- Implement a switch or if-else structure to map incoming Bluetooth characters (e.g., 'S' for Sweep) to servo movements.
- Connect the ESP32 to the computer via USB, select the correct COM port in the Arduino IDE, and upload the sketch.

4.3 Program Operation

- Press the Enable (EN) button on the ESP32 to start the program. Open the Serial Monitor at the set baud rate to confirm the "Bluetooth initialized" status.

- Open a Bluetooth Terminal app on a smartphone and pair it with the ESP32 device name defined in the code.
- Observe the smartphone terminal to see the real-time values sent from the ESP32 sensor.
- Type and send the pre-defined commands from the smartphone app to trigger the servo motor's movement.

4.4 Coding

```
#include "BluetoothSerial.h"
#include <ESP32Servo.h>
#include "DHT.h"

// Define Pin
#define DHTPIN 27
#define DHTTYPE DHT11
#define SERVO_PIN 18
#define LED 2

BluetoothSerial SerialBT;
DHT dht(DHTPIN, DHTTYPE);
Servo myServo;

char command;

void setup() {
  Serial.begin(9600);
  SerialBT.begin("Mechamato");
  Serial.println("Bluetooth initialized... Ready to receive commands.");

  dht.begin();
  myServo.attach(SERVO_PIN);
  pinMode(LED, OUTPUT);
}

void loop() {
```

```

// 1. Read data from the sensor
float h = dht.readHumidity();

// 2. Send the data to the Phone
if (isnan(h)) {
    Serial.println("Fail to read DHT11!");
} else {
    SerialBT.print("Humidity: ");
    SerialBT.print(h);
    SerialBT.println("%");

    Serial.print("Humidity: ");
    Serial.print(h);
    Serial.println("%");
}

// 3. Received command from the phone
if (SerialBT.available()) {
    command = SerialBT.read();

    if (command == 'S') { //press S to start
        myServo.write(180); // rotate 90 degree
        digitalWrite(LED, HIGH);
        SerialBT.println("Servo moved to 90!");
    }
    else if (command == 'R') { // press 'R' to reset
        myServo.write(0);
        digitalWrite(LED, LOW);
        SerialBT.println("Servo reset to 0!");
    }
}

delay(2000);
}

```

5.0 DATA COLLECTION

Sent Command (Smartphone)	Servo Action	Sensor Reading (on Smartphone)	System Status/Remarks
'S'	360° Anti-Clockwise	Humidity: 53%	Command 'S' triggered continuous CCW rotation.
'R'	360° Clockwise	Humidity: 53%	Command 'R' reversed direction to CW.
(No command)	Continue last rotation	Humidity: 53%	Data updates every 2s as per code delay.

6.0 DATA ANALYSIS

The data analysis for this task confirms the successful integration of bidirectional Bluetooth communication using the ESP32's 2.4 GHz radio frequency and Frequency-Hopping Spread Spectrum (FHSS) technology. When the 'S' or 'R' commands were sent from the smartphone terminal, the ESP32 successfully received these packets via the Bluetooth Classic Serial Port Profile (SPP). This triggered the switch logic in the firmware, resulting in the continuous 360° rotation of the servo motor—anti-clockwise for 'S' and clockwise for 'R'. The hardware responded to these asynchronous serial commands as a virtual wireless bridge, mimicking a physical wired UART connection.

Simultaneously, the system demonstrated reliable remote monitoring by transmitting real-time sensor data from the hardware to the mobile device. While the servo was in motion, the smartphone terminal consistently displayed a humidity level of 53%, indicating that the DHT22 sensor was being sampled and the data was being successfully encapsulated into Bluetooth packets for transmission. The use of `SerialBT.print()` allowed the sensor values to be reassembled accurately by the mobile terminal app. The consistent 53% reading confirms

that the environmental data remained stable throughout the actuation trials, and the two-second delay in the loop ensured that the Bluetooth stream was not overwhelmed by data packets.

7.0 RESULTS

In these experiments, we use Esp32 and monitor the system output through the apps in our phone. At first, the Esp32 we used successfully connected with our phone's bluetooth and connected with the application in our phone. After that, the DHT 11 sensor provided the humidity sensor reading every 2 seconds and it was transmitted to the mobile application through "SerialBT.print". Furthermore, we also can control the servo motor through the application in the phone. When the character 'S' is sent in the application, it will cause the servo to rotate to the 180 degrees position. While the character 'R' is sent in the application, it will cause the servo to rotate to the 0 degrees position.

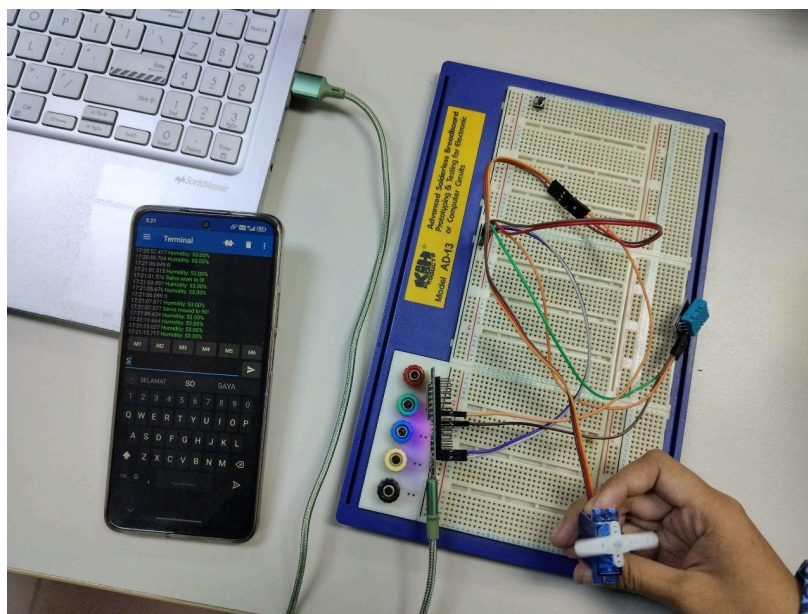


Diagram 7.1 show the sensor is detecting the room's humidity and display it in the phone.

8.0 DISCUSSION

How the bluetooth can corporate into the iot application is by allowing different devices to communicate and exchange data wirelessly over short distances. It allows the users to control home appliances, like lights or fans, using a smartphone app without using any cables. After that, the users also can monitor the sensor at the real time if the system has a built-in sensor in the system. The bluetooth connects smart devices together, so they can work automatically.

For the experiment, we use “myServo.write(180)” and “myServo.write(0)” but when we executed the experiment, the servo will keep moving and not stop at certain angle because the servo we use a 360-degree Continuous Rotation Servo. Thus the servo either moves clockwise or anti-clockwise at continuous motion only. Other than that, we delay at the DHT11 function. The “delay(2000)” is necessary because the DHT11 is a slow sensor that requires at least 1-2 seconds between reads.

9.0 CONCLUSION

In conclusion, this project successfully demonstrated the development of a remote monitoring and control system by integrating an ESP32 with Bluetooth technology to facilitate seamless, wireless data exchange. By leveraging the ESP32's built-in 2.4 GHz radio frequency and Frequency-Hopping Spread Spectrum (FHSS), the system maintained a reliable communication link, effectively preventing interference by switching between channels. The ESP32 functioned as a virtual wireless bridge using the Bluetooth Classic Serial Port Profile (SPP), which allowed for the real-time transmission of sensor data to a smartphone while simultaneously receiving asynchronous serial commands to actuate physical components. Experimental results confirmed the system's efficacy, as humidity data from the DHT sensor was consistently monitored at a stable 53%, and command inputs 'S' and 'R' successfully triggered the servo motor's 360° rotation in both anti-clockwise and clockwise directions. Ultimately, the integration of these components proves that Bluetooth is a robust tool for Mechatronics and IoT applications, providing a versatile interface for both remote data acquisition and hardware manipulation.

10.0 RECOMMENDATIONS

The recommendation for this experiment is that we can change the "delay(2000)" function with "millis()" function. This would allow the Bluetooth commands to be processed instantly without waiting for the 2-second sensor timer to finish. Other than that, we can implement an automatic moving servo using the sensor. For example, servo moves to 0 degree if the humidity is low, the servo moves to 90 degrees if the humidity is medium and lastly the servo moves to 180 degrees if the humidity is high. Other than that, we also can use an external power source like a 18650 battery and a buck converter specifically for the Servo to avoid interfering with the ESP32's logic stability.

11.0 REFERENCES

Arduino Bluetooth Basic Tutorial. (n.d.). Arduino Project Hub.

<https://projecthub.arduino.cc/NeilChaudhary/arduino-bluetooth-basic-tutorial-9cff12>

Dejan. (2022, February 18). *Arduino and HC-05 Bluetooth module complete tutorial*. How to Mechatronics.

<https://howtomechatronics.com/tutorials/arduino/arduino-and-hc-05-bluetooth-module-tutorial/>

Basic Bluetooth communication with Arduino & HC-05. (n.d.). Arduino Project Hub.

<https://projecthub.arduino.cc/superturis/basic-bluetooth-communication-with-arduino-hc-05-3a431c>

ACKNOWLEDGEMENT

We would like to sincerely express our gratitudes to our instructors, Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli Bin Zainal and the lab technician, for the help given during the task. The knowledge and suggestions are able to guide us throughout the progress of the task. We also would like to thank our classmates for their support and help which really give positive impacts to the progression of our task.

STUDENTS' DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been untaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed

unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

Signature: 

Read [/]

Name: NOR EFFENDI BIN ANUAR @ ANWAR

Understand [/]

Matric Number: 2310519

Agree [/]

Signature: 

Read [/]

Name: NUR AINA SAFFIYA BINTI MOHD TARMIZI

Understand [/]

Matric Number: 2411644

Agree [/]

Signature: 

Read [/]

Name: NUR ATIKAH BINTI KAMARUL BAHARIN

Understand [/]

Matric Number: 2319846

Agree [/]